Guía de Clase: Contenedores con Docker

Universidad del Quindío - Infrastructure Computing Course

Duración: 2 horas (120 minutos)
Nivel: Intermedio
Prerrequisitos: Conocimientos básicos de Linux, red y sistemas operativos

---

Objetivos de Aprendizaje

Al finalizar esta sesión, el estudiante será capaz de:

1. Explicar qué son los contenedores y su diferencia con las máquinas virtuales
2. Crear y gestionar imágenes y contenedores con Docker
3. Escribir un Dockerfile funcional
4. Orquestar aplicaciones multi-contenedor con Docker Compose
5. Desplegar una aplicación en un entorno contenedorizado

Tabla de Contenidos

- [0:00 - 0:15] Introducción teórica
- [0:15 - 0:30] Instalación y primeros comandos
- [0:30 - 0:55] Creación de imágenes con Dockerfile
- [0:55 - 1:05] Pausa activa
- [1:05 - 1:30] Docker Compose en la práctica
- [1:30 - 1:50] Ejercicio integrador
- [1:50 - 2:00] Cierre y evaluación

Sección 1: Introducción Teórica (15 minutos)

¿Qué es un contenedor?

Un contenedor es una unidad de software empaquetada que incluye código, dependencias y configuraciones. Ejecuta una aplicación de forma aislada pero comparte el kernel del sistema operativo huésped.

Contenedores vs Máquinas Virtuales

Característica	Máquina Virtual	Contenedor
Peso	Gigabytes	Megabytes
Inicio	Minutos	Segundos
Aislamiento	Hardware virtualizado	Procesos del OS

Característica	Máquina Virtual	Contenedor
Overhead	Alto (OS completo)	Bajo (comparte kernel)
Portabilidad	Limitada	Alta (define/run)

Casos de uso en la industria

- Desarrollo local consistente (elimina "en mi máquina funciona")
- CI/CD pipelines
- Microservicios
- Despliegue en la nube (AWS ECS, GCP GKE, Azure AKS)

Sección 2: Instalación y Primeros Comandos (15 minutos)

Verificación de instalación

```
docker --version
docker info
```

Comandos esenciales

1. Ejecutar un contenedor hello-world

```
docker run hello-world
```

2. Buscar imágenes en Docker Hub

```
docker search ubuntu
docker search nginx
docker search node
```

3. Descargar y ejecutar un contenedor interactivo

```
docker pull ubuntu:22.04
docker run -it ubuntu:22.04 /bin/bash
# Dentro del contenedor:
lsb_release -a
exit
```

4. Gestionar contenedores

```
# Listar contenedores (activos e inactivos)
docker ps          # solo activos
docker ps -a       # todos

# Iniciar/detener un contenedor
docker start <container_id>
docker stop <container_id>
docker rm <container_id>

# Ejecutar en modo detached (fondo)
docker run -d --name mi-servidor nginx
docker logs mi-servidor
docker exec -it mi-servidor /bin/bash
```

5. Publicar puertos

```
# Mapear puerto local 8080 al puerto 80 del contenedor
docker run -d -p 8080:80 --name web nginx
# Acceder desde navegador: http://localhost:8080
```

Sección 3: Creación de Imágenes con Dockerfile (25 minutos)

¿Qué es un Dockerfile?

Es un archivo de texto con instrucciones que Docker lee para construir una imagen automáticamente.

Estructura básica

```
# Imagen base
FROM ubuntu:22.04

# Metadatos
LABEL maintainer="profesor@unquindio.edu.co"
LABEL version="1.0"

# Variables de entorno
ENV APP_PORT=3000
ENV NODE_ENV=production

# Instalar dependencias
RUN apt-get update && apt-get install -y \
    curl \
    wget \
    && rm -rf /var/lib/apt/lists/*

# Copiar archivos al contenedor
```

```
WORKDIR /app
COPY . /app

# Exponer puertos
EXPOSE 3000

# Comando de inicio
CMD ["node", "server.js"]
```

Instrucciones principales

Instrucción	Descripción
`FROM`	Define la imagen base
`RUN`	Ejecuta comandos durante la build
`COPY`	Copia archivos del host al contenedor
`ADD`	Similar a COPY con soporte de URLs/tar
`WORKDIR`	Establece el directorio de trabajo
`ENV`	Define variables de entorno
`EXPOSE`	Documenta el puerto que usa la app
`CMD`	Comando por defecto al ejecutar
`ENTRYPOINT`	Comando principal (menos flexible)
`ARG`	Variable de build-time
`VOLUME`	Crea un mount point para datos
`HEALTHCHECK`	Define cómo verificar salud

Buenas prácticas

1. **Usar imágenes oficiales** (Docker Official Images)
2. **Minimizar capas** con `&&` en RUN
3. **Usar .dockerignore** para excluir archivos innecesarios
4. **Multi-stage builds** para reducir tamaño final
5. **No usar root** cuando sea posible
6. **Etiquetar imágenes** con versión, no `latest`

Ejemplo: Multi-stage build

```
# Stage 1: Build
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
```

```
COPY . .
RUN npm run build

# Stage 2: Runtime
FROM nginx:alpine
COPY --from=builder /app/dist /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Ejercicio práctico: Crear una imagen

Archivo: `server.js`

```
const http = require('http');
const os = require('os');

const server = http.createServer((req, res) => {
    res.writeHead(200, {'Content-Type': 'application/json'});
    res.end(JSON.stringify({
        hostname: os.hostname(),
        uptime: os.uptime(),
        message: 'Hola desde la Universidad del Quindío!'
    }));
});

server.listen(3000, () => {
    console.log('Servidor corriendo en puerto 3000');
});
```

Archivo: `Dockerfile`

```
FROM node:20-alpine
WORKDIR /app
COPY server.js .
EXPOSE 3000
CMD ["node", "server.js"]
```

Construir y ejecutar:

```
# Construir la imagen
docker build -t unquindio/app:1.0 .

# Verificar la imagen
docker images | grep unquindio

# Ejecutar
docker run -d -p 3000:3000 --name app-demo unquindio/app:1.0
```

```
# Probar
curl http://localhost:3000

# Ver logs
docker logs app-demo
```

Sección 4: Docker Compose (25 minutos)

¿Qué es Docker Compose?

Herramienta para definir y ejecutar aplicaciones multi-contenedor con un archivo YAML.

Comandos principales

```
docker compose up        # Iniciar servicios
docker compose down      # Detener y remover
docker compose ps        # Estado de servicios
docker compose logs -f   # Logs en tiempo real
docker compose exec <servicio> <comando>
```

Ejemplo: Aplicación web con base de datos

Archivo: `docker-compose.yml`

```
version: '3.8'

services:
  web:
    build: .
    ports:
      - "3000:3000"
    environment:
      - DATABASE_HOST=db
      - DATABASE_PORT=5432
      - DATABASE_NAME=appdb
    depends_on:
      - db
    networks:
      - app-network
    restart: unless-stopped

  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: secret123
      POSTGRES_DB: appdb
```

```
    volumes:
      - db-data:/var/lib/postgresql/data
    networks:
      - app-network
    restart: unless-stopped

  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"
    networks:
      - app-network
    restart: unless-stopped

volumes:
  db-data:

networks:
  app-network:
    driver: bridge
```

Ejercicio: Desplegar con Compose

```
# Iniciar todos los servicios
docker compose up -d

# Verificar estado
docker compose ps

# Ver logs de un servicio específico
docker compose logs -f web

# Acceder a la base de datos
docker compose exec db psql -U admin -d appdb

# Verificar red
docker compose exec web ping db

# Detener servicios
docker compose down

# Detener y eliminar volúmenes
docker compose down -v
```

Sección 5: Ejercicio Integrador (20 minutos)

Scenario: Desplegar un sistema de monitoreo básico

Requisitos:

1. API REST que reporte métricas del sistema
2. Base de datos PostgreSQL para almacenar datos
3. Redis como caché
4. Interfaz web (Nginx) como reverse proxy
5. Todo orquestado con Docker Compose

Paso 1: Estructura del proyecto

```
monitoring-system/
├── api/
│   ├── Dockerfile
│   ├── server.js
│   └── package.json
├── nginx/
│   ├── Dockerfile
│   └── nginx.conf
├── docker-compose.yml
└── .dockerignore
```

Paso 2: API (api/server.js)

```
const http = require('http');
const os = require('os');

const server = http.createServer((req, res) => {
    const cpuUsage = Math.round(os.loadavg()[0] * 100);
    const memTotal = os.totalmem();
    const memFree = os.freemem();
    const uptime = Math.floor(os.uptime() / 60);

    if (req.url === '/health') {
        res.writeHead(200, {'Content-Type': 'application/json'});
        res.end(JSON.stringify({status: 'healthy', timestamp: new
Date().toISOString()}));
        return;
    }

    res.writeHead(200, {'Content-Type': 'application/json'});
    res.end(JSON.stringify({
        hostname: os.hostname(),
        platform: os.platform(),
        cpuUsage: `${cpuUsage}%`,
        memoryTotal: `${(memTotal / 1073741824).toFixed(2)} GB`,
        memoryFree: `${(memFree / 1073741824).toFixed(2)} GB`,
        uptimeMinutes: uptime,
        cpus: os.cpus().length,
        timestamp: new Date().toISOString()
    }));
});
```

```
server.listen(3001, '0.0.0.0', () => {
    console.log('API de monitoreo en puerto 3001');
});
```

Paso 3: Dockerfile de la API (api/Dockerfile)

```
FROM node:20-alpine
WORKDIR /app
COPY package.json .
RUN npm install --production
COPY server.js .
EXPOSE 3001
CMD ["node", "server.js"]
```

Paso 4: Nginx config (nginx/nginx.conf)

```
upstream api_backend {
    server api:3001;
}

server {
    listen 80;
    server_name localhost;

    location / {
        root /usr/share/nginx/html;
        index index.html;
    }

    location /api/ {
        proxy_pass http://api_backend/;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }

    location /health {
        proxy_pass http://api_backend/health;
    }
}
```

Paso 5: docker-compose.yml

```
version: '3.8'
```

```
services:
  api:
    build: ./api
    expose:
      - "3001"
    networks:
      - monitoring
    healthcheck:
      test: ["CMD", "wget", "--spider", "-q",
"http://localhost:3001/health"]
      interval: 30s
      timeout: 10s
      retries: 3
    restart: unless-stopped

  nginx:
    build: ./nginx
    ports:
      - "80:80"
    depends_on:
      - api
    networks:
      - monitoring
    restart: unless-stopped

networks:
  monitoring:
    driver: bridge
```

Paso 6: Ejecutar y probar

```
# Construir y ejecutar
docker compose up -d --build

# Verificar servicios
docker compose ps

# Probar API directamente
curl http://localhost/api/

# Probar health check
curl http://localhost/health

# Ver logs en tiempo real
docker compose logs -f

# Ver métricas del sistema
curl http://localhost/api/ | jq
```

Sección 6: Cierre y Evaluación (10 minutos)

Conceptos clave a retener

1. **Imágenes** = plantillas inmutables (como clases en OOP)
2. **Contenedores** = instancias de imágenes (como objetos)
3. **Dockerfile** = receta para construir imágenes
4. **Docker Compose** = orquestación de múltiples contenedores
5. **Volúmenes** = persistencia de datos
6. **Redes** = comunicación entre contenedores

Comandos más usados (cheat sheet)

```
# Imágenes
docker build -t <nombre>:<tag> .
docker images
docker rmi <image_id>

# Contenedores
docker run -d -p <host>:<container> --name <name> <image>
docker ps -a
docker stop/start/rm <container>
docker exec -it <container> <command>

# Logs y debugging
docker logs <container>
docker inspect <container>
docker stats

# Compose
docker compose up -d
docker compose down
docker compose logs -f <service>
```

Preguntas de repaso

1. ¿Cuál es la diferencia entre `CMD` y `ENTRYPOINT` en un Dockerfile?
2. ¿Por qué es importante usar `.dockerignore`?
3. ¿Cómo persistir datos entre reinicios de contenedores?
4. ¿Qué hace la instrucción `EXPOSE` en un Dockerfile?
5. ¿Cómo comunicarse entre dos contenedores en el mismo compose?

Tarea para la casa

Desarrollar una aplicación full-stack simple (frontend + backend + base de datos) con Docker Compose que incluya:

- Al menos 3 servicios
- Volumen persistente para la base de datos

- Red personalizada
- Health checks
- Variables de entorno para configuración

Recursos Adicionales

- Documentación oficial: https://docs.docker.com
- Docker Hub: https://hub.docker.com
- Dockerfile best practices: https://docs.docker.com/develop/develop-images/dockerfile_best-practices/
- Docker Compose reference: https://docs.docker.com/compose/compose-file/

Guía elaborada para la cátedra de Infraestructura Computacional
Universidad del Quindío - 2026